

```
# school - > admission -> name, age, gender , class

def greet(name, age ):  # single parameter
    print(f"your entered name is {name} :)")
    print(f"you are {age} years old. pls verify! ")
    # print(f" Your gender is : {gender}")
    # print(f"the class you have applied is : {cls}")

user_name = input("enter your name: ")
user_age = int(input("enter your age: "))
# user_gender = input("enter your gender: ")
# user_cls = input("enter the class that you are applying for! ")

greet(age = user_age, name = user_name) # function call
```

```
enter your name: ganesh
enter your age: 21
your entered name is ganesh :)
you are 21 years old. pls verify!
```

```
def add_return(a, b):

    print(a + b)

add_return(10, 20)
```

```
30
```

```
# print() vs return()

def add(sub1,sub2):  # func creation
    return(sub1+sub2)

total = add(10,5)

print(total)
```

```
15
```

```
# =====
#                               PYTHON FUNCTIONS
# =====

# WHY DO WE NEED FUNCTIONS?
#
# Suppose we want to print "Welcome to Python!"
# 100 times.
#
# Without functions:

print("Welcome to Python!")
print("Welcome to Python!")
print("Welcome to Python!")

# Imagine writing this 100 times.
#
# It makes the program:
#
# ✓ Longer
# ✓ Harder to Read
# ✓ Difficult to Maintain
#
# Functions solve this problem.
#
# A Function is a reusable block of code
# that performs a specific task.

# =====
# Function Syntax
# =====
```

```

# def function_name():    #creating the function
#     code
#
# def                -> Keyword used to create a function
#
# function_name      -> Name of the function # user defined
#
# ()                 -> Parentheses
#
# :                  -> Start of the function body


# =====
# Example 1
# Creating and Calling a Function
# =====

def greet():

    print("Welcome to Python!")

# Creating a function DOES NOT execute it.

# To execute it,
# we CALL the function.

greet()

# Output
#
# Welcome to Python!


# =====
# Example 2
# Calling Multiple Times
# =====

greet()
greet()
greet()

# Output
#
# Welcome to Python!
# Welcome to Python!
# Welcome to Python!

# Notice:
#
# We wrote the code ONCE
# but used it many times.


# =====
# Example 3
# Function with One Parameter
# =====

def greet(name):

    print("Welcome to python", name)

# name is called a PARAMETER.
#
# A parameter receives the value
# passed to the function.

greet(name )

greet("Alice")

greet("Rahul")

# Output
#
# Welcome John
# Welcome Alice
# Welcome Rahul

```

```

# =====
# Parameter vs Argument
# =====

# Parameter
#
# Variable inside the function definition.

def hello(name,age):

    print(name)

# name is a PARAMETER.

# Argument
#
# Actual value passed
# when calling the function.

hello("John",21)

# "John" is an ARGUMENT.

# =====
# Example 4
# Multiple Parameters
# =====

def student(name, age):

    print("Name :", name)

    print("Age :", age)

student("John", 21)

# Output
#
# Name : John
# Age : 21

# =====
# Example 5
# print() vs return()
# =====

# ----- print() -----

def add_print(a, b):
    print(a + b)

result = add_print(10, 20) # 30

print(result)

# Output
#
# 30
# None
#
# Why?
#
# print() only displays the value.
#
# It DOES NOT send the value back.
#
# Every function without return
# automatically returns None.

# ----- return() -----

def add_return(a, b):

    return(a + b)

result = add_return(10, 20)

```

```

print(result)

# Output
#
# 30
#
# return sends the value
# back to the caller.

# =====
# Example 6
# Returning Values
# =====

def square(number):

    return number * number

answer = square(5)

print(answer)

# Output
#
# 25

# =====
# Example 7
# Function with if-else
# =====

def even_or_odd(number):

    if number % 2 == 0:

        return "Even"

    else:

        return "Odd"

print(even_or_odd(10))

print(even_or_odd(7))

# Output
#
# Even
# Odd

# =====
# Example 8
# Function with Loop
# =====

def print_numbers():

    for i in range(1, 6):

        print(i)

print_numbers()

# Output
#
# 1
# 2
# 3
# 4
# 5

# =====
# Example 9
# Function with User Input
# =====

def add_numbers():

```

```

a = int(input("Enter First Number : "))

b = int(input("Enter Second Number : "))

print("Sum =", a + b)

# Uncomment to test

# add_numbers()

# =====
# Example 10
# Default Parameters
# =====

# A default parameter provides
# a value automatically
# if the user doesn't pass one.

def greet(name="Guest"):

    print("Welcome", name)

greet()

greet("John")

# Output
#
# Welcome Guest
# Welcome John

# =====
# Example 11
# Keyword Arguments
# =====

def employee(name, salary):

    print("Name :", name)

    print("Salary :", salary)

# Positional Arguments

employee("John", 50000)

# Keyword Arguments

employee(salary=50000, name="John")

# Output
#
# Name : John
# Salary : 50000
#
# Since parameter names are mentioned,
# order doesn't matter.

# =====
# Example 12
# Returning Multiple Values
# =====

def calculate(a, b):

    return a+b, a-b

sum_value, difference = calculate(10, 5)

print(sum_value)

print(difference)

# Output
#
# 15
# 5

```

```

# =====
# Example 13
# Function Calling Another Function
# =====

def square(x):

    return x * x

def cube(x):

    return square(x) * x

print(cube(5))

# Output
#
# 125


# =====
# Example 14
# *args (Arbitrary Positional Arguments)
# =====

*args

# Normally

def add(a,b):
    print(a+b)

# accepts only TWO values.
#
# What if we don't know
# how many values the user will send?
#
# Python provides *args.
#
# *args accepts ANY NUMBER
# of positional arguments.
#
# Python stores them
# inside a TUPLE.

def add(*args):

    print(args)

add(10,20)

add(10,20,30)

add(10,20,30,40,50)

# Output
#
# (10,20)
#
# (10,20,30)
#
# (10,20,30,40,50)

# Loop through args

def total(*numbers):

    answer = 0

    for number in numbers:

        answer += number

    return answer

print(total(10,20))

print(total(10,20,30))

```

```

print(total(10,20,30,40))

# Output
#
# 30
# 60
# 100
#
# Think of *args as:
#
# "I don't know how many values
# the user will give me.
# Just collect all of them."

```

```

# add
# def addition(a,b):
#     print(a+b)

# *args

def addition(*numbers):
    count = 0
    for i in numbers:
        count = count + i
    print(count)

addition(1,2,3,4,5,6,102)

```

123

```

def student_data(**data):
    for key,value in data.items():
        print(f"{key}: {value}")

student_data(

    name="John",

    age=21,

    city="Chennai"

)

*ARGS    -> tuple
**kwargs -> Dictionary

```

```

name: John
age: 21
city: Chennai

```

```

# =====
# Example 15
# **kwargs (Arbitrary Keyword Arguments)
# =====

# Until now we passed values like this:
#
# student("John", 21)
#
# These are POSITIONAL arguments.
#
# But sometimes we want to pass
# values using parameter names.
#
# Example:
#
# student(name="John", age=21)
#
# These are called KEYWORD arguments.
key -> value
#
# What if we don't know how many
# keyword arguments the user will pass?
#
# Python provides **kwargs.
#

```

```

# **kwargs accepts ANY NUMBER
# of keyword arguments.
#
# Python stores them
# inside a DICTIONARY.

def student(**kwargs):
    print(kwargs)

def addition(*numbers):
    print(numbers)

student(

    name="John",

    age=21,

    city="Chennai"

)

# Output
#
# {
# 'name': 'John',
# 'age': 21,
# 'city': 'Chennai'
# }

# =====
# Accessing kwargs
# =====

def student(**details):

    print(details["name"])

    print(details["age"])

student(

    name="John",

    age=21

)

# Output
#
# John
# 21

# =====
# Looping through kwargs
# =====

def student(**details):

    for key, value in details.items():

        print(key, ":", value)

student(

    name="John",

    age=21,

    city="Delhi",

    course="Data Science"

)

# Output
#
# name : John
# age : 21
# city : Delhi

```



```

# course : Data Science

# =====
# Real World Example of kwargs
# =====

# Imagine creating user profiles.
#
# One user enters:
#
# Name
# Age
#
# Another enters:
#
# Name
# Age
# Email
# Phone
# Address
#
# Instead of changing the function
# every time,
#
# **kwargs can collect everything.

def create_user(**user):

    print(user)

create_user(

    name="Alice",

    age=25,

    email="alice@gmail.com",

    city="Bangalore"

)

# =====
# *args vs **kwargs
# =====

# *args
#
# Collects POSITIONAL arguments.
#
# Stored as a TUPLE.

def demo(*args):

    print(args)

demo(10,20,30)

# Output
#
# (10,20,30)

# **kwargs
#
# Collects KEYWORD arguments.
#
# Stored as a DICTIONARY.

def demo(**kwargs):

    print(kwargs)

demo(

    name="John",

    age=21

```

```

)

# Output
#
# {
#   'name': 'John',
#   'age': 21
# }

# =====
# Using *args and **kwargs Together
# =====

def information(*args, **kwargs):

    print("ARGS :", args)

    print("KWARGS :", kwargs)

information(

    10,

    20,

    30,

    name="John",

    age=21,

    city="Delhi"

)

# Output
#
# ARGS :
# (10,20,30)
#
# KWARGS :
# {
#   'name': 'John',
#   'age': 21,
#   'city': 'Delhi'
# }

# =====
# LAMBDA FUNCTION
# =====

# Lambda is an ANONYMOUS FUNCTION.
#
# Anonymous means:
#
# It has NO NAME.
#
# It is generally used when
# we need a small function
# only once.
#
# Normal Function

def square(x):

    return x * x

print(square(5))

# Output
#
# 25

# Same thing using Lambda

square = lambda x: x * x

print(square(5))

```

```
# Output
#
# 25

# =====
# Lambda Syntax
# =====

# lambda arguments : expression

#
# lambda
#
# Keyword
#
# arguments
#
# Input values
#
# expression
#
# Single line of code
#
# Lambda ALWAYS returns the result.
#
# There is NO need to write
# the return keyword.

# =====
# Lambda Example 2
# =====

add = lambda a, b: a + b

print(add(10,20))

# Output
#
# 30

# =====
# Lambda Example 3
# =====

multiply = lambda a, b: a * b

print(multiply(5,6))

# Output
#
# 30

# =====
# Lambda Example 4
# =====

cube = lambda x: x ** 3

print(cube(4))

# Output
#
# 64

# =====
# Lambda Example 5
# =====

even = lambda x: x % 2 == 0

print(even(10))

print(even(7))

# Output
#
# True
```

```
# False

# =====
# Why Lambda?
# =====

# Imagine you only need a function
# ONE TIME.
#
# Instead of writing
#
# def
# return
#
# we simply write
#
# lambda

# =====
# Normal Function vs Lambda
# =====

# Normal Function

def square(x):
    return x*x
print(square(5))

# Lambda

square = lambda x:x*x

print(square(5))

# Both produce
#
# 25

# =====
# When Should We Use Lambda?
# =====

# Use NORMAL FUNCTIONS
#
# ✓ Multiple lines of code
# ✓ Complex logic
# ✓ Reusable code
#
#
# Use LAMBDA
#
# ✓ One-line functions
# ✓ Temporary functions
# ✓ Mostly with map(), filter(), sorted()

# =====
# Practice Questions
# =====

# 1. Create a function to print your name.
#
# 2. Create a function that returns
#    the square of a number.
#
# 3. Create a function with
#    default parameters.
#
# 4. Create a function that accepts
#    three parameters.
#
# 5. Create a function using
#    keyword arguments.
#
# 6. Create a function using
#    *args to add numbers.
#
# 7. Create a function using
```

```

# **kwargs to print student details.
#
# 8. Create a lambda function
#    to calculate the cube.
#
# 9. Create a lambda function
#    to check whether
#    a number is positive.
#
# 10. Create a lambda function
#     to multiply two numbers.

# =====
# SUMMARY
# =====

# def
# ----
# Used to create functions.
#
#
# Parameters
# -----
# Variables in function definition.
#
#
# Arguments
# -----
# Values passed while calling.
#
#
# return
# -----
# Sends a value back.
#
#
# Default Parameters
# -----
# Used when no value is passed.
#
#
# Keyword Arguments
# -----
# Pass values using parameter names.
#
#
# *args
# ----
# Accepts ANY NUMBER
# of positional arguments.
#
# Stored as Tuple.
#
#
# **kwargs
# -----
# Accepts ANY NUMBER
# of keyword arguments.
#
# Stored as Dictionary.
#
#
# Lambda
# -----
# Anonymous function.
#
# One line.
#
# Automatically returns the result.

```

```

# *args, **kwargs, lambda function + 3 project

def add(*numbers, **kwargs):    # ( 10 , 20, 20, , , )
    count = 0
    for number in numbers:
        count = count + number

    print(count)

```

```
add(1,2,3,4,5,6,6,7,8,8,9,23,123,12,3,12312, name = "amirht ")
```

```
12532
```

```
string = "python"
```

```
print(string)
```

```
python
```

```
print(*string)
```

```
p y t h o n
```

Start coding or generate with AI.